Comprehensive Project: Classification with Statistical Validation

Network Traffic Analysis and Intrusion Detection using CIC-IDS2017

Christian Gómez Magdaleno
Noe Israel Felipe Perez
Andrea Guadalupe Garcia Oñate
University of Guanajuato

April 14, 2026

Abstract

In this work, a complete multi-class classification solution oriented toward cybersecurity is developed using the CIC-IDS2017 dataset. The primary objective is to evaluate and compare the performance of two algorithms (XGBoost and Random Forest) through an experimental design based on repeated stratified cross-validation. Statistical hypothesis tests are applied to determine the significance of the observed differences.

1 Problem Definition

The dataset used is **CIC-IDS2017** (Canadian Institute for Cybersecurity), which contains captures of benign network traffic and the most up-to-date cyberattacks. After a cleaning and auditing process that mitigated multicollinearity, null values, zero variance, and inconsistencies generated by the feature extractor, the final dataset consists of numerical attributes.

The problem is framed as a **multi-class classification**, where the target variable (*Label*) aims to identify whether the traffic is legitimate or belongs to a specific attack category.

Context and Impact: The operational decision supported by this model is the automated management of traffic and the generation of alerts in a real-time Intrusion Detection System (IDS). In this domain, a *False Positive* (blocking a legitimate user) generates temporary operational friction, but a *False Negative* (failing to detect an attack) can compromise the confidentiality, integrity, and availability of critical infrastructure.

Addressing this problem through **multi-class classification** allows us to better represent reality and is fundamental for Security Operations Center (SOC) operations. Precisely identifying the attack topology allows for:

- **Differentiated Incident Response:** Mitigation strategies vary by threat. A Distributed Denial of Service (DDoS) attack requires routing rules and load balancing on the network, while a Brute Force attack demands IP blocking at the firewall level and credential rotation.
- **Prioritization (Triage):** Not all attacks have the same urgency. A port scan (*PortScan*) is usually low-priority exploratory noise, but a network infiltration (*Infiltration*) or web application attacks (*Web Attack*) represent active breaches requiring immediate human attention.
- **Threat Intelligence and Forensics:** Correct labeling of the attack type allows defensive teams to understand the adversary's Tactics, Techniques, and Procedures, facilitating the patching of the root vulnerability (e.g., repairing a database after detecting SQL injections).

2 Metric Selection and Justification

The **CIC-IDS2017** dataset has a massive class imbalance. Under this scenario, the *Accuracy* metric is misleading; a model could ignore all minority classes and still obtain a score close to 100%, failing in its fundamental purpose of detection.

To mitigate this bias, the **Macro F1-Score** has been selected as the primary metric.

2.1 Macro F1-Score

The F1-Score is the harmonic mean between precision and recall. Unlike the arithmetic mean, the harmonic mean penalizes extreme values, ensuring that a model must perform well in both metrics to obtain a high F1:

$$F1_i = 2 \cdot \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i}$$

For a multi-class classification problem with N categories, there are two main variants: *Micro* (weighted by the number of instances) and *Macro* (simple average). This project uses the **Macro** approach, formally defined as:

$$F1_{Macro} = \frac{1}{N} \sum_{i=1}^{N} F1_i$$

Justification: By calculating the unweighted average of individual $F1$ scores, we treat each attack category with equal importance, regardless of its frequency. This is critical in cybersecurity, as an Infiltration attack is extremely rare but potentially devastating; the F1-Macro ensures the model is severely penalized if it fails to detect these minority classes, forcing the algorithms (XGBoost and Random Forest) to learn representative decision boundaries for the entire threat spectrum.

3 Model Selection

To address the dimensional complexity and non-linear relationships of network traffic, two architectures based on tree ensembles were selected:

3.1 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized implementation of the *Gradient Boosting* algorithm. Unlike parallel ensemble methods, XGBoost builds trees sequentially. Each new tree attempts to predict the residual errors (the difference between the actual value and the accumulated prediction) of the previous trees.

Technical Detail and Assumptions:

- **Objective Function:** It minimizes a loss function that includes an L1 and L2 regularization term to prevent overfitting:

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

  where $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penalizes model complexity.

- **Advantages:** Native handling of missing values, support for distributed computing and GPU, and tree pruning based on negative gain.

Algorithm 1: XGBoost Pseudocode

Input: Dataset $\mathcal{D}$, Loss function L, Learning rate η
Output: Final Model $F(x)$

```
Initialize F_0(x) = arg min_γ Σ_{i=1}^{n} L(y_i, γ);
for m = 1 to M do
    Calculate gradients and Hessians of the loss function;
    Train a base tree f_m(x) using the residuals of model F_{m-1}(x);
    Update the model: F_m(x) = F_{m-1}(x) + η · f_m(x);
end
```

3.2 Random Forest

Random Forest is a supervised learning algorithm that utilizes the *Bagging* (Bootstrap Aggregating) technique. It builds a "forest" of uncorrelated decision trees trained on different subsets of the data.

Technical Detail and Assumptions:

- **Randomness Injection:** In addition to using random row samples (*Bootstrapping*), it selects a random subset of features at each node split. This reduces model variance without significantly increasing bias.

- **Advantages:** Excellent for handling class imbalance (using `class_weight`) and providing an intrinsic measure of variable importance.

- **Assumptions:** It is assumed that individual trees have sufficient depth to capture patterns, relying on averaging to smooth out the error.

Algorithm 2: Random Forest Pseudocode

```
Input: Dataset 𝒟, Number of trees B
Output: Tree ensemble G(x)
for b = 1 to B do
    Take a Bootstrap sample 𝒟_b from the original dataset;
    Train a tree T_b on 𝒟_b;
        At each node, randomly select k features from the total p features;
        Choose the best split among the k features;
end
Prediction: Average probabilities (regression) or majority voting
  (classification);
```

4 Statistical Testing

4.1 Normality Test (Shapiro-Wilk)

The **Shapiro-Wilk** test calculates a W statistic, which measures the correlation between observed data and theoretical values from a normal distribution:

$$W = \frac{\left(\sum_{i=1}^{n} a_i x_{(i)}\right)^2}{\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Where $x_{(i)}$ are the ordered data points and a_i are coefficients based on the expected values of order statistics of a normal distribution. A W value close to 1.0 indicates high adherence to normality.

Hypothesis Statement:

- H_0: Performance differences follow a normal distribution ($p > 0.05$).
- H_1: Performance differences **do not** follow a normal distribution ($p \leq 0.05$).

This test acts as a methodological filter: if H_0 is accepted, the *Paired Student's t-test* is used; otherwise, the non-parametric *Wilcoxon test* is chosen, ensuring the study's conclusions are statistically valid.

4.2 Paired Student's t-Test

If the normality assumption is satisfied, the **Paired Student's t-test** is employed. The test evaluates whether the mean of the differences ($\bar{d}$) between the two models is significantly different from zero. The t statistic is calculated as:

$$t = \frac{\bar{d}}{s_d/\sqrt{n}}$$

Where:

- $\bar{d}$: Average of the differences between XGBoost and Random Forest scores.
- s_d: Standard deviation of those differences.
- n: Total number of runs ($n = 30$).

The t value is contrasted with a theoretical distribution with $n - 1$ degrees of freedom to obtain the p-value.

Interpretation:

- If $p \leq 0.05$: H_0 is rejected, concluding that one model is superior to the other with a 95% confidence level.
- If $p > 0.05$: It is concluded that the observed differences are due to chance and no statistical superiority exists between the algorithms.

5 Experimental Design

5.1 Data Partitioning

Prior to any training phase, the dataset was divided stratifically. **15% was reserved as a test set (*Test Set*)**, completely isolated, which served as the final evaluation on unseen data. On the remaining 85%, the cross-validation process was carried out (80% training and 20% validation).

5.2 Preprocessing and Scaling

All numerical variables were normalized using **Standardization (*StandardScaler*)**. This process adjusted the distribution of each variable to have a mean of 0 and a standard deviation of 1, stabilizing the mathematical learning process.

5.3 Repeated Stratified Cross-Validation

On the training set, a **Repeated Stratified Cross-Validation** (*Repeated Stratified K-Fold*) design was implemented. A configuration of $k = 5$ folds repeated 6 times was used, generating a total of **30 independent runs** per model. Stratification ensured that each iteration maintained the exact proportion of the original classes, allowing for stable and comparable performance distributions for subsequent statistical analysis.

5.4 Hyperparameter Tuning

An empirical search for hyperparameters was conducted to configure the models for optimal performance and adequate generalization.

Table 1: Hyperparameter Configuration

Model	Hyperparameter	Final Value
XGBoost	*n_estimators*	150
	learning_rate (η)	0.08
	max_depth	6
	subsample	0.8
	colsample_bytree	0.8
	max_delta_step	1
	reg_alpha ($L1$)	1
	reg_lambda ($L2$)	2.5
	gamma	0.2
Random Forest	*n_estimators*	200
	max_depth	17
	class_weight	balanced
	min_samples_split	10
	min_samples_leaf	4

6 Statistical Analysis

The **Shapiro-Wilk** test was applied to evaluate the normality assumption. Since the data presented a normal distribution ($p = 0.3403 > 0.05$), the **Paired Student's t-test** was used.

The results yielded a statistic of 15.7672 and a p-value of 9.2141×10^{-16}, which is significantly lower than the significance level $\alpha = 0.05$. Consequently, the **null hypothesis (H_0) is rejected**, concluding that there is a statistically significant difference between the models.

7 Results Evaluation

Below are the descriptive metrics extracted after executing the experimental design for the Macro F1-Score over 30 iterations.

Table 2: Detailed F1-Score (Macro) Results per Execution

	XGBoost			Random Forest		
Ex.	**Train**	**Val**	**Test**	**Train**	**Val**	**Test**
1	0.8825	0.8249	0.8246	0.7890	0.7375	0.7530
2	0.8453	0.8269	0.8082	0.7679	0.7576	0.7358
3	0.8499	0.8346	0.8152	0.7574	0.7109	0.7298
4	0.8608	0.8262	0.8118	0.8313	0.8061	0.7556
5	0.8512	0.8082	0.8101	0.7745	0.7360	0.7442
6	0.8449	0.7874	0.7943	0.7666	0.7226	0.7348
7	0.8530	0.8173	0.8031	0.7661	0.7519	0.7353
8	0.8816	0.8298	0.8359	0.7733	0.7580	0.7429
9	0.8844	0.8712	0.8325	0.8275	0.7871	0.7509
10	0.8488	0.8160	0.8013	0.8454	0.7491	0.7684
11	0.8889	0.8374	0.8297	0.7881	0.7486	0.7495
12	0.8898	0.8651	0.8307	0.8521	0.7939	0.7957
13	0.8891	0.8544	0.8240	0.7991	0.7865	0.7556
14	0.8700	0.8275	0.8274	0.7819	0.7300	0.7326
15	0.8487	0.8233	0.8065	0.7761	0.7460	0.7407
16	0.8781	0.8565	0.8265	0.8043	0.7561	0.7526
17	0.8472	0.8196	0.8076	0.8258	0.7803	0.7712
18	0.8560	0.8342	0.8194	0.7826	0.7442	0.7429
19	0.8668	0.8219	0.8127	0.7712	0.7357	0.7395
20	0.8628	0.8026	0.8206	0.8140	0.7848	0.7426
21	0.8453	0.8000	0.8052	0.7625	0.7271	0.7347
22	0.8491	0.8205	0.7923	0.7799	0.7525	0.7376
23	0.8929	0.8282	0.8383	0.7915	0.7600	0.7497
24	0.8538	0.8338	0.8159	0.7860	0.7384	0.7459
25	0.8862	0.8348	0.8118	0.8185	0.7827	0.7489
26	0.8842	0.8406	0.8208	0.7760	0.7370	0.7331
27	0.8901	0.8784	0.8254	0.7847	0.7471	0.7441
28	0.8421	0.8447	0.7987	0.7766	0.7474	0.7390
29	0.8783	0.8383	0.8219	0.8206	0.8046	0.7588
30	0.8935	0.8198	0.8285	0.7637	0.7139	0.7318

Table 3: Descriptive Statistics for Validation Macro F1-Score over 30 Iterations

Statistical Metric	**XGBoost**	**Random Forest**
Minimum	0.7874	0.7109
Maximum	0.8784	0.8061
Median	0.8279	0.7489
Average (μ)	0.8308	0.7545
Standard Deviation (σ)	0.0199	0.0252

8 Visualization

To support the descriptive analysis, the dispersion and stability of both algorithms are illustrated through boxplots and error ranking charts.

boxplot_name.png

Figure 1: Boxplot of the Macro F1-Score distribution across the 30 iterations.

ranking_name.png

Figure 2: Average ranking of models with error margins (Standard Deviation).

9 Detailed Analysis by Model

9.1 XGBoost

Below are the detailed results of the global validation performance ($F1 \approx 0.8279$), corresponding to the median run.

Table 4: Detailed Classification Report: XGBoost

Class	**Precision**	**Recall**	**F1-Score**	**Support**
BENIGN	1.00	1.00	1.00	340,881
Bot	0.42	0.99	0.59	295
DDoS	1.00	1.00	1.00	19,204
DoS GoldenEye	0.96	1.00	0.98	1,544
DoS Hulk	1.00	1.00	1.00	34,519
DoS Slowhttptest	0.89	0.99	0.94	825
DoS slowloris	0.98	0.99	0.99	869
FTP-Patator	1.00	1.00	1.00	1,191
Heartbleed	1.00	1.00	1.00	2
Infiltration	0.60	0.60	0.60	5
PortScan	0.99	1.00	1.00	23,839
SSH-Patator	0.95	1.00	0.97	885
Web Attack – Brute Force	0.71	0.65	0.68	226
Web Attack – Sql Injection	0.22	0.67	0.33	3
Web Attack – XSS	0.25	0.52	0.34	98
Macro Average	**0.80**	**0.89**	**0.83**	**424,386**
Weighted Average	**1.00**	**1.00**	**1.00**	**424,386**

confusion_matrix_xgboost_median.png

Figure 3: Confusion Matrix: XGBoost Median in Validation

Table 5: Global Macro F1-Score Performance - XGBoost

Dataset	**F1-Score (Macro)**
Training (Train)	0.8700
Validation (Val)	0.8275
Testing (Test)	0.8274

9.2 Random Forest

Below are the detailed results of the global validation performance ($F1 \approx 0.7491$), corresponding to the median run.

Table 6: Detailed Classification Report: Random Forest

Class	**Precision**	**Recall**	**F1-Score**	**Support**
BENIGN	1.00	0.98	0.99	340,881
Bot	0.07	0.99	0.13	295
DDoS	0.99	1.00	1.00	19,204
DoS GoldenEye	0.97	1.00	0.98	1,544
DoS Hulk	0.93	1.00	0.96	34,519
DoS Slowhttptest	0.94	0.99	0.96	825
DoS slowloris	0.99	0.99	0.99	869
FTP-Patator	0.99	1.00	0.99	1,191
Heartbleed	1.00	1.00	1.00	2
Infiltration	1.00	0.20	0.33	5
PortScan	0.99	1.00	1.00	23,839
SSH-Patator	0.92	1.00	0.96	885
Web Attack – Brute Force	0.52	0.46	0.49	226
Web Attack – Sql Injection	0.50	0.67	0.57	3
Web Attack – XSS	0.09	0.86	0.17	98
Macro Average	**0.79**	**0.87**	**0.77**	**424,386**
Weighted Average	**0.99**	**0.98**	**0.99**	**424,386**

confusion_matrix_rf_median.png

Figure 4: Confusion Matrix: Random Forest Median in Validation

Table 7: Global Macro F1-Score Performance - Random Forest

Dataset	F1-Score (Macro)
Training (Train)	0.8454
Validation (Val)	0.7491
Testing (Test)	0.7684

10 Discussion

The analysis reveals an overfitting phenomenon in both models, manifested in the gap between training performance and validation/testing results. This behavior is primarily rooted in the nature of the dataset and the sensitivity of the selected metrics.

10.1 Impact of Class Imbalance and F1-Macro

The dataset presents extreme imbalance: the *BENIGN* class represents approximately 80% of the total with 2.2 million records, while critical categories such as *Heartbleed*, *SQL Injection*, and *Infiltration* have only 11, 21, and 36 records, respectively.

By using **Macro F1-Score** as the primary metric, equal statistical weight is given to each of the 15 classes without considering their support. In this scenario, the model easily learns the patterns of the majority class (achieving an F1 close to 1.0) but fails to generalize on minority classes. The observed overfitting does not necessarily imply memorization of general noise but rather memorization of the few instances of scarce classes present in training. When faced with new records of these same classes in the test set, the model loses predictive capacity, sinking the global average.

10.2 Hyperparameter Optimization

During experimentation, it was observed that the models persistently tended to overfit. The empirical hyperparameter search slightly mitigated this trend; however, it was not enough to eliminate the generalization gap.

For future work, empirical search could be replaced by formal optimization methods such as **Grid Search CV** or **Randomized Search CV**. These strategies, by systematically evaluating the search space through cross-validation, could identify configurations that penalize model complexity more rigorously, favoring more robust architectures.

10.3 Suggested Mitigation Strategies

To reduce the impact of imbalance and improve generalization capacity, the following technical strategies are proposed:

- **Resampling Techniques:** Implement *Synthetic Minority Over-sampling Technique* (SMOTE) to generate synthetic examples for low-support classes or *Random Under Sampling* to balance the dominant class.

- **Class Grouping (Re-labeling):** Consolidate minority classes with similar characteristics into macro-categories. For example, group the various *Web Attack* variants (Brute Force, XSS, SQL Injection) into a single class named *Web Attacks*. This would increase the resulting class's support, facilitating the detection of common patterns.

- **Cost-Sensitive Learning:** Configure algorithms to assign a higher weight to errors committed on minority classes during the loss function, compensating for numerical disparity.

- **Strict Regularization:** Increase penalty parameters (such as *lambda* and *alpha* in XGBoost, or reducing maximum depth and increasing minimum samples per leaf in Random Forest) to force the model to learn more global patterns rather than memorizing specific instances.

Finally, a critical limitation detected resides in the high computational cost of processing millions of records, suggesting that any future optimization should consider the use of specialized hardware or stratified representative sampling techniques.

11 Conclusions

Following the analysis of the data obtained in the 30 iterations of the experimental design, it is concluded that the XGBoost model, under the described conditions, presents a statistically different performance compared to Random Forest. The results of the paired Student's t-test confirm that this difference is significant, with a p-value < 0.05, allowing for the rejection of the null hypothesis of equal means.

Additionally, the XGBoost model exhibited a consistently higher Macro F1-Score compared to Random Forest in both the validation and test sets. This trend remained stable throughout the runs, demonstrating that the boosting tree architecture captures the complexities of the CIC-2017 dataset more effectively than the bagging ensemble approach. Ultimately, the statistical evidence supports that XGBoost offers superior generalization capability for the classification of malicious traffic under the evaluated metric.